

### Frage 1

(1 Punkt)

Welche drei *Phasen* werden beim **Test-Driven-Development** durchlaufen?

- A. red, green, refactor
- B. given, when, then
- C. arrange, act, assert

**Antwort** (eine gültige Antwort):

### Frage 2

(1 Punkt)

Welche der folgenden **Listentypen** wäre am passendsten zur Speicherung von Listenelementen geeignet, wenn ein Listenelement nicht mehr als einmal vorkommen darf, und auch der Wert **null** ebenfalls nicht eingefügt werden darf?

- A. Collection
- B. List
- C. Set
- D. Map
- E. Vector

**Antwort** (eine gültige Antwort):

### Frage 3

(2 Punkte)

Welches sind gültige **Eigenschaften** der Klasse **TreeSet** aus dem *Collections* Framework?

- A. benutzt Schlüsselwerte zur Speicherung von Werten
- B. bewahrt die "natürliche" Ordnung der eingefügten Elemente
- C. gestattet das Hinzufügen eines 'null' Elementes
- D. erlaubt das Einfügen von Duplikaten

**Antwort** (mehrere gültige Antworten möglich):

#### Frage 4

(1 Punkt)

Was bedeutet **Überladung** (von Methoden)?

- A. Wenn eine Methode eine Methode der Elternklasse übersteuert
- B. Wenn eine Methode mehr als 3 Argumente aufweist
- C. Wenn mehrere Methoden gleichen Names existieren, aber mit unterschiedlichen Parametern bzw. Parameterlisten

**Antwort** (eine gültige Antwort):

#### Frage 5

(3 Punkte)

Java kennt unterschiedliche sogenannte **Access Modifier**, die den **Zugriff** bzw. die **Sichtbarkeit** von Klassen, Datenattributen oder Methoden steuern. Nennen Sie diese und beschreiben Sie kurz deren Bedeutung!

**Antwort:**

### Frage 6

(3 Punkte)

Setzen Sie die erforderlichen **Schlüsselworte** im folgenden kleinen Klassenmodell ein!

```
public interface Observable {}  
public interface Train _____ Observable {}  
public abstract class AbstractTrain _____ Train {}  
public class LongDistanceTrain _____ AbstractTrain {}
```

Antwort:

### Frage 7

(3 Punkte)

Was ist ein **Interface**? Beschreiben Sie die *Bedeutung* und nennen sie mindestens 2 Beispiele für deren *Eigenschaften*.

Antwort:

## Frage 8

(1 Punkt)

Der nachfolgende Unit-Test wird nicht funktionieren. Warum nicht?

```
package dhbw.exam;

public class Person {
    private String name;
    private String getName() { return this.name; }
    public void setName(String name) { this.name = name; }
}
```

```
package dhbw.tests;

public class PersonTests {
    @Test
    public void canGetName() {
        // given
        Person person = new Person();

        // when
        String name = person.getName();

        // then
        assertNotNull(name);
    }
}
```

**Antwort:**

### Frage 9

(2 Punkte)

Was ist der Unterschied zwischen **impliziter** und **expliziter Umwandlung** von primitiven Datentypen? Nennen Sie je ein Beispiel.

**Antwort:**

### Frage 10

(1 Punkt)

Welche der drei Aussagen trifft bzgl. des Schlüsselwortes **static** zu? (der Code-Ausschnitt unten dient als Ansichtsbeispiel)

- A. Der Aufruf einer statischen Methode kann nur über eine Instanz der umgebenden Klasse erfolgen
- B. Der Wert einer statischen Klassenvariable kann nicht geändert werden
- C. Der Aufruf einer statischen Methode erfordert keine Instanz der umgebenden Klasse

```
public class DateConverter {  
    public static String TIME_ZONE = "UTC";  
    public static Date parse(String dateAsString) { ... }  
}
```

**Antwort** (eine gültige Antwort):

### Frage 11

(2 Punkte)

Was ist der Unterschied zwischen **Aggregation** und **Komposition**?

**Antwort:**

### Frage 12

(2 Punkte)

Welche der folgenden Elemente spezifizieren Assoziation zwischen Objekten? (*mehrere gültige Antworten*)

**Antwort** (*mehrere gültige Antworten möglich*):

- A. Besitz
- B. Relation
- C. n:m
- D. 1:1
- E. HTTP
- F. Vererbung

### Frage 13

(2 Punkte)

Was bedeutet die folgende Klassensignatur?

```
public class ShortMessage<T extends Message> { ... }
```

**Antwort:**

### Frage 14

(2 Punkte)

Schreibe eine generische **Methode** (**Methodensignatur** reicht) auf, die die folgenden Anforderungen erfüllt. Die Methode ...

- soll zwei Zahlen addieren können
- soll generisch sein
- der generische Typ soll auf Zahlen beschränkt sein
- soll generische Zahlen als Eingabewerte entgegennehmen
- soll geschützt sein
- gibt das Additionsergebnis generisch zurück

**Antwort:**

### Frage 15

(3 Punkte)

Was ist ein **funktionales Interface** und nennen sie (mindestens) 2 aus dem Bereich der Lambda's?

Antwort:

### Frage 16

(2 Punkte)

Was ist ein **Predicate**?

Antwort:

### Frage 17

(1 Punkt)

Welches **funktionale Interface** wird im folgenden Code-Ausschnitt genutzt?

```
UnaryOperator<Integer> functionalInterface = (Integer x) -> {  
    return x * x;  
};  
Integer result = functionalInterface.apply(5);
```

Antwort:

### Frage 18

(2 Punkte)

Gegeben sei die folgende Aufzählung. Implementiere eine beispielhafte **switch** Anweisung für die Elemente dieses **enum**s.

```
public enum Character {  
    PALADIN, WIZARD, ROGUE;  
}
```

Antwort:

### Frage 19

(1 Punkt)

Erläutere kurz die **Bedeutung** der Methode **ordinal()** bei **enum**s?

```
public enum WeekDay {
```

```
MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY;  
}
```

Welchen konkreten **Wert** gibt der folgende Aufruf zurück?

```
? = WeekDay.FRIDAY.ordinal();
```

**Antwort:**

## Frage 20

(2 Punkte)

Was ist die **Bedeutung** des Ausdrucks `checked exception`?

**Antwort:**

## Frage 21

(2 Punkte)

Was ist der Unterschied zwischen `Error` und `Exception` in Java?

**Antwort:**

## Frage 22

(1 Punkt)

Was ist der Zweck des Schlüsselwortes `throws` in der folgenden Methodensignatur?

```
public void risky() throws Exception {...}
```

- A. Wird benutzt, um mögliche Fehler bei der Ausführung der Methode anzuzeigen, und der Fehler muss nicht behandelt werden
- B. Wird benutzt, um mögliche Fehler bei der Ausführung der Methode anzuzeigen, und der Fehler muss behandelt werden
- C. Die Methode wirft in jedem Fall eine Ausnahme, sie muss behandelt werden

**Antwort** (eine gültige Antwort):

## Frage 23

(2 Punkte)



Wie ist ein **Fehler** im Quellcode zu behandeln?

- A. Mithilfe eines 'try' Statements
- B. Mithilfe eines 'try-catch' Statements
- C. Mithilfe eigener Implementierungen, die auftretende Fehler verarbeiten
- D. Mithilfe eines 'try-handle' Statements
- E. Mithilfe eines 'try-catch-finally' Statements

**Antwort** (mehrere gültige Antworten möglich):